AffordanceBench Studio: An Open Toolkit for Query Processing, Evaluation, and Visualization in 3D Multimedia Affordance Research

Hao Jiang
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
h-jiang24@mails.tsinghua.edu.cn

Kaiwen Deng
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
dkw24@mails.tsinghua.edu.cn

Weihong Li
Tsinghua Shenzhen International Graduate School, Tsinghua University
Shenzhen, China
li-wh24@mails.tsinghua.edu.cn

Jinhao Chen
School of Management and Engineering, Nanjing University
Nanjing, China
502024150012@smail.nju.edu.cn

Wenming Yang
Shenzhen International Graduate School and Department of Electronic Engineering, Tsinghua University
Shenzhen, China
yang.wenming@sz.tsinghua.edu.cn

Abstract

Research on 3D affordance understanding increasingly mixes point clouds, textual queries, benchmark-specific protocols, and figure-oriented analysis, yet many research codebases remain difficult to build, inspect, and reuse. We present *AffordanceBench Studio*, an open toolkit that organizes query processing, dataset-facing entrypoints, benchmark-style evaluation, visualization, profiling, and release-oriented reproducibility utilities for 3D multimedia affordance research. The contribution is not a new affordance model; it is a reviewable software package that reduces engineering friction for researchers who need to run query-conditioned affordance workflows, evaluate public datasets, export summaries, and construct lightweight reproducibility pipelines. The current release provides a unified command-line interface, a legacy bridge for integrating existing research scripts, environment validation, documentation-first quickstart material, and a public-safe release discipline that avoids bundling private paths, embedded secrets, or under-review main-paper artifacts by default. By emphasizing buildability, command discoverability, compatibility, and software packaging discipline, the toolkit is designed to make multimedia affordance workflows easier to inspect, reuse, and demonstrate.

Keywords

3D affordance research, multimedia software, open-source toolkit, query processing, evaluation workflows, reproducibility

1 Introduction

3D multimedia affordance research increasingly depends on the interaction between geometry, language, benchmark protocols, and figure-oriented analysis. In practice, however, these workflows are often spread across research-only scripts, machine-specific launch commands, and undocumented evaluation paths. The result is a familiar reproducibility problem: even when the underlying code exists, external users struggle to determine which script to run, which environment variables to set, which datasets are expected, and how to move from raw outputs to the artifacts needed for inspection or demonstration.

This software-layer problem is becoming more visible as affordance research expands from benchmark-centric 3D understanding pipelines such as 3D AffordanceNet [3] to open-vocabulary detection [7, 9, 10], language-guided segmentation [6], and more language-heavy reasoning systems [2, 12]. In many of these workflows, transferable priors from large-scale vision-language pretraining, e.g., CLIP, PointCLIP, and ULIP [8, 11, 13], also become part of the engineering stack that users must reproduce correctly.

Recent multimedia OSS papers further suggest that reusable evaluation and experimentation infrastructure can itself be a first-class contribution when packaged as a reviewable toolkit rather than a loose artifact bundle, e.g., VLMEvalKit, OpenDIC, and CLaM [1, 4, 5]. We therefore position *AffordanceBench Studio* explicitly as software infrastructure for query-conditioned affordance workflows, not as a second method paper.

AffordanceBench Studio addresses this software-layer gap by packaging reusable research infrastructure for query-conditioned 3D affordance workflows. The toolkit organizes dataset-facing entrypoints, evaluation helpers, lightweight visualization utilities, profile-oriented scripts, and release-facing documentation into a single software project with a unified command-line interface. The current public release bridges to legacy research scripts while presenting a cleaner public-facing surface for installation, command discovery, dry-run inspection, and environment checking.

The ACM Multimedia Open Source Software track rewards packages that are broad, buildable, demoable, and externally inspectable. We therefore make a direct software claim: this submission contributes reusable infrastructure for query-conditioned affordance workflows, not merely a cleaned-up code release around one paper result.

Our main contribution is a toolkit that turns an internal research script collection into a more buildable and reviewable software package. In its current form, the toolkit already exposes **29 task-oriented commands** across training, evaluation, query utilities, visualization, profiling, and packaging operations.

More concretely, the current release contributes three things:

- a **public core** with a unified CLI, packaging metadata, documentation, reproducibility notes, and submission-oriented materials;
- a **legacy bridge** that makes existing research scripts discoverable through a shared command registry instead of isolated machine-specific launch commands;
- a **public-safe release discipline** that improves openness while deliberately avoiding embedded secrets, private paths, and default release of under-review main-paper artifacts.

2 Toolkit Overview

Figure 1 shows the intended first-pass review path: diagnose a clean machine, inspect a resolved command chain, generate a concrete query artifact, and inspect a released output/evidence composite. This is stronger than a code dump because it makes the package inspectable before full setup and concrete after execution.

Under the hood, the toolkit adopts a two-layer architecture. The contribution is the **public core**: a unified CLI, command registry, path-resolution helpers, documentation, a Dockerfile, packaging metadata, and submission-oriented materials that make heterogeneous workflows installable, inspectable, and reviewable. The **legacy bridge** is the compatibility mechanism: it preserves existing research scripts and shell workflows while exposing them through the public command layer without a risky full rewrite.

The current command registry groups workflows into six categories:

- **core:** environment checking and command discovery;
- **train:** training entrypoints for full and prompt-only variants;
- **eval:** risk subset evaluation, boundary-sensitive metrics, interaction-proxy analysis, and fairness-oriented comparisons;
- **LASO/query:** question-as-query evaluation, zero-shot mappings, prompt translation, and anchor-map construction;
- **visualization/profile:** heatmaps, t-SNE plots, failure cases, sensitivity curves, efficiency profiling, and stage breakdown;
- **ops:** Google Drive download helpers, backup manifests, and batch shell workflows.

This command structure lowers first-pass review cost. A reviewer can run an environment check, inspect available commands, inspect one command with a human-readable description, and only then execute a target workflow. That sequence communicates maturity better than a single entry script.

3 Buildability, Compatibility, and Usage

A core design principle of the toolkit is to prefer explicit configuration over hidden machine-specific assumptions. The public command layer standardizes reviewer-facing entrypoints around environment variables such as `OPENAD_BASE` and `LASO_ROOT`, keeps private absolute paths out of the public workflow, and routes external-service credentials through environment variables rather than embedded secrets.

The toolkit also improves software compatibility by supporting both Python and shell-based workflows through a shared command registry. Users can inspect resolved commands with a dry-run mode before executing them, which is especially useful in mixed CPU/GPU environments. In addition, an `env-check` command verifies whether key files, CLIP dependencies, and dataset entrypoints are available before the user launches heavier workflows. This "early failure" behavior is important for software quality because it turns implicit assumptions into readable error conditions, and the current release has already been exercised both on a remote Linux host and in a clean local virtual environment.

Table 1: Representative review-friendly workflows already supported by the current toolkit release.

Goal	Representative command path
Setup	`env-check` → `list` → `describe`
Query path	`laso-anchor-map` → `laso-qaq`
Output path	`render-heatmap` / `visualize-tsne`
Release prep	`backup-manifest` → `package-backup-assets`

The current release package includes:

- a Python package entrypoint (`affordbench`);
- packaging metadata (`pyproject.toml`);
- an installable dependency list (`requirements.txt`);
- a Dockerfile for container-oriented setup;
- quickstart, architecture, command docs, and repro notes;
- submission materials: review alignment, project page template, and OpenReview checklist.

Taken together, these elements make the package buildable, inspectable, and reviewable as software. The command registry and documentation layer also make the system legible in a short simulation-first demo instead of only in repository archaeology.

Representative command patterns already supported by the current release include:

- `affordbench env-check` for early environment validation;
- `affordbench describe <command>` for command discovery and dry inspection;
- LASO-facing commands for question-as-query evaluation and prompt processing;
- visualization commands for heatmaps, t-SNE, failure cases, and sensitivity curves;
- profiling commands for latency and stage-breakdown analysis.

4 Representative Use Cases

Environment onboarding. The package turns setup diagnosis into a first-class workflow: `env-check`, then `list`, then `describe`. This replaces guesswork with inspectable pre-execution checks.

Query-conditioned experiment setup. LASO-facing prompt preparation, anchor-map generation, and benchmark-style evaluation live under one command registry. The software gain is lower iteration cost, not more script sprawl.

Artifact generation. The toolkit exports reviewer-readable artifacts, not only checkpoints. Heatmaps, t-SNE plots, failure cases, and sensitivity curves are exposed as callable commands.

Release hygiene. Backup manifests and split-archive helpers make packaging a supported workflow rather than a last-minute cleanup task. This is software maturity, not repository ornament.

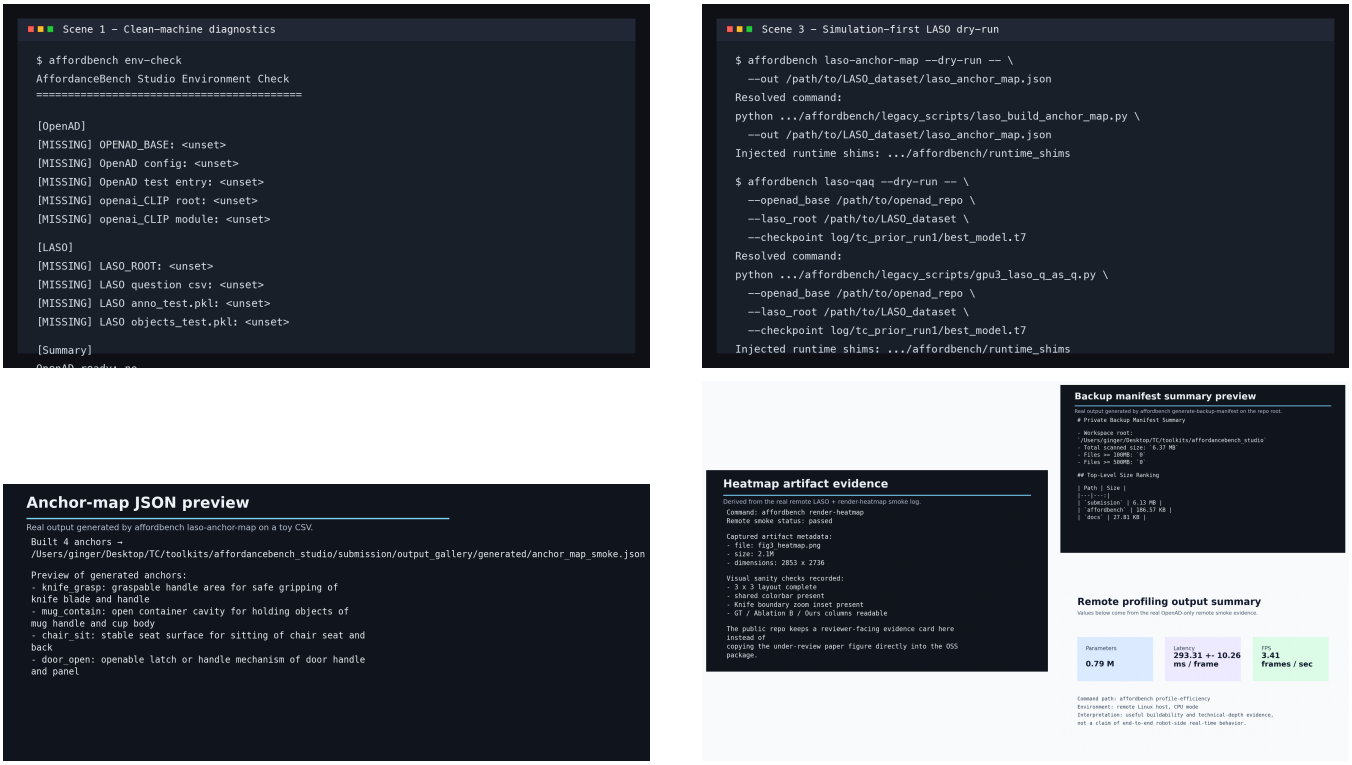

Figure 1: Reviewer-facing evidence panel from the public release. Top row: clean-machine diagnostics and dry-run command resolution make the package inspectable before full dataset setup. Bottom left: a real anchor-map JSON preview. Bottom right: a paper-oriented output/evidence composite built only from released assets, bundling backup-manifest, heatmap-evidence, and profiling-summary cards.

5 Review Fit and Demonstration Path

This submission should be judged as reviewable software, not merely as released code. The decisive evidence is concrete and reviewer-facing: buildability, compatibility, demo suitability, and real outputs are shown rather than merely claimed.

The same evidence supports a compact demonstration path: `env-check` → `describe laso-anchor-map` → `laso-anchor-map`. This yields a concrete JSON artifact of the kind shown in Figure 1. Heavier profiling and heatmap paths are backed by released remote smoke evidence. That sequence gives reviewers both low-friction inspection and at least one concrete output.

6 Intended Audience and Extension Path

The audience is concrete: researchers adapting query-conditioned affordance workflows and reproducibility teams spanning setup, evaluation, and packaging. It also serves demo builders needing a stable CLI. The extension claim is equally concrete: contributors can register a legacy workflow, document assumptions, and expose it through the public command layer without rewriting the stack. For a fast-moving affordance ecosystem, that upgrade path is more valuable than a rigid monolith.

Table 2: Reviewer objections and the strongest evidence already available in the public release.

Reviewer question	Answer backed by released evidence
"Author-machine only?"	No. Clean-venv install evidence shows the package installs outside the authors' existing shell state.
"Fresh-clone reviewable?"	Yes. The public-main fresh-clone path proves that `main` can be cloned, installed, and inspected in a new directory.
"Tagged archive still works?"	Yes. The source-ZIP install path validates the release artifact itself rather than only a live checkout.
"Any real execution?"	Yes. Remote OpenAD smoke confirms a real Linux host can execute a lightweight profiling-oriented path.
"Any concrete outputs?"	Yes. Remote LASO + heatmap smoke confirms a real query-plus-visualization artifact chain beyond dry-run resolution.

7 Release Discipline, Availability, and Limitations

The toolkit targets research software release, not a full paper-cycle artifact dump. The current public-safe release discipline therefore excludes large benchmark datasets, embedded secrets or private service tokens, and under-review main-paper weights or final figures. This boundary reduces overlap risk with concurrent research submissions while also making the package cleaner and easier to inspect.

The current public release already includes a permissive license, packaging metadata, a Dockerfile, quickstart docs, and submission support material. That package is already reviewable as software: it can be downloaded, built, documented, and demonstrated through public artifacts rather than private instructions. The public project page is available at chilled-watermelon.github.io/affordancebench-studio, the source repository is available at github.com/Chilled-watermelon/affordancebench-studio, and a source ZIP snapshot is available at `release source ZIP (v0.1.2)`.

Representative use cases include: (1) quickly validating a machine setup before running an affordance workflow; (2) launching LASO-style query experiments without reconstructing bespoke path conventions; (3) exporting visualization and profiling artifacts for reports or demonstrations; and (4) organizing large-file packaging and reproducibility manifests for future release preparation. These use cases emphasize broad applicability and practical engineering value rather than a single benchmark headline.

The toolkit still has limitations. It does not package large public datasets by default, full benchmark reproduction still requires users to prepare datasets and checkpoints on their own systems, and some commands still rely on legacy internals. Those constraints reflect a deliberate release boundary: prioritize buildability, inspectability, and incremental extension over a risky late-stage rewrite.

8 Community Value and Maintenance Plan

The package has operational value beyond the current command set. By separating a public-facing CLI and documentation layer from the legacy implementation layer, it becomes easier to extend, audit, and maintain across future affordance benchmarks. A small research group can use it as a reproducibility scaffold; a more engineering-oriented team can use it as a base for demos, packaging, and internal forks with clearer upgrade paths.

9 Conclusion

AffordanceBench Studio is an open toolkit for query processing, evaluation, visualization, profiling, and reproducibility-oriented packaging in 3D multimedia affordance research. Its contribution is a reviewable software package: a unified CLI, a compatibility-preserving bridge over legacy workflows, public build evidence, and a simulation-first inspection path. The release therefore fits the ACM Multimedia Open Source Software track as buildable, inspectable, demoable, and extensible software infrastructure rather than as a research-code dump.

References

[1] Xiaodong Chen, Kunlang He, Wu Liu, Xinchen Liu, Zheng-Jun Zha, and Tao Mei. 2024. CLaM: An Open-Source Library for Performance Evaluation of Text-driven Human Motion Generation. In *Proceedings of the 32nd ACM International Conference on Multimedia*. Association for Computing Machinery, New York, NY, USA, 11194–11197. doi:10.1145/3664647.3685523

[2] Hengshuo Chu, Xiang Deng, Qi Lv, Xiaoyang Chen, Yinchuan Li, Jianye Hao, and Liqiang Nie. 2025. 3D-AffordanceLLM: Harnessing Large Language Models for Open-Vocabulary Affordance Detection in 3D Worlds. In *The Thirteenth International Conference on Learning Representations*. OpenReview.net, Singapore, 17 pages.

[3] Shengheng Deng, Xun Xu, Chaozheng Wu, Ke Chen, and Kui Jia. 2021. 3D AffordanceNet: A Benchmark for Visual Object Affordance Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 1778–1787. doi:10.1109/CVPR46437.2021.00182

[4] Haodong Duan, Junming Yang, Yuxuan Qiao, Xinyu Fang, Lin Chen, Yuan Liu, Xiaoyi Dong, Yuhang Zang, Pan Zhang, Jiaqi Wang, et al. 2024. VLMEvalKit: An Open-Source Toolkit for Evaluating Large Multi-Modality Models. In *Proceedings of the 32nd ACM International Conference on Multimedia*. Association for Computing Machinery, New York, NY, USA, 11198–11201. doi:10.1145/3664647.3685520

[5] Wei Gao, Huiming Zheng, Chenhao Zhang, Kaiyu Zheng, Zhuozhen Yu, Yuan Li, Hua Ye, and Yongchi Zhang. 2024. OpenDIC: An Open-Source Library and Performance Evaluation for Deep-learning-based Image Compression. In *Proceedings of the 32nd ACM International Conference on Multimedia*. Association for Computing Machinery, New York, NY, USA, 11202–11205. doi:10.1145/3664647.3685519

[6] Yicong Li, Na Zhao, Junbin Xiao, Chun Feng, Xiang Wang, and Tat-seng Chua. 2024. LASO: Language-guided Affordance Segmentation on 3D Object. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 14251–14260. doi:10.1109/CVPR52733.2024.01351

[7] Toan Nguyen, Minh Nhat Vu, An Vuong, Dzung Nguyen, Thieu Vo, Ngan Le, and Anh Nguyen. 2023. Open-Vocabulary Affordance Detection in 3D Point Clouds. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, IEEE, Piscataway, NJ, USA, 5692–5698. doi:10.1109/IROS55552.2023.10341553

[8] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*. PMLR, PMLR, Virtual Event, 8748–8763.

[9] Tuan Van Vo, Minh Nhat Vu, Baoru Huang, Toan Nguyen, Ngan Le, Thieu Vo, and Anh Nguyen. 2024. Open-Vocabulary Affordance Detection using Knowledge Distillation and Text-Point Correlation. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, IEEE, Piscataway, NJ, USA, 13968–13975.

[10] Lin Wu, Wei Wei, Peizhuo Yu, and Jianglin Lan. 2025. Open-Vocabulary 3D Affordance Understanding with Functional Text Enhancement and Multilevel Representation Alignment. In *Proceedings of the 33rd ACM International Conference on Multimedia*. Association for Computing Machinery, New York, NY, USA, 7988–7997. doi:10.1145/3746027.3755239

[11] Le Xue, Mingfei Gao, Chen Xing, Roberto Martín-Martín, Jiajun Wu, Caiming Xiong, Ran Xu, Juan Carlos Niebles, and Silvio Savarese. 2023. ULIP: Learning a Unified Representation of Language, Images, and Point Clouds for 3D Understanding. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 1179–1189. doi:10.1109/CVPR52729.2023.00121

[12] Chunlin Yu, Hanqing Wang, Ye Shi, Haoyang Luo, Sibei Yang, Jingyi Yu, and Jingya Wang. 2025. SeqAfford: Sequential 3D Affordance Reasoning via Multimodal Large Language Model. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 1691–1701. doi:10.1109/CVPR52733.2025.00164

[13] Renrui Zhang, Ziyu Guo, Wei Zhang, Kunchang Li, Xupeng Meng, Yuheng Chen, Hao Peng, and Peng Gao. 2022. PointCLIP: Point Cloud Understanding by CLIP. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, Piscataway, NJ, USA, 8552–8562. doi:10.1109/CVPR52688.2022.00838